PlugPlan SG: Coding Agent Reflections

SC2006 · Lab 4, section 3.3.4 · Author: Team SCE3-04 · Drafted with Claude Code · 10 November 2026

Assistant used for both exercises: Claude Code (model Claude Opus 5.5), started as a fresh agent with no memory of the earlier work.

Reflection 1 (3.3.2): Regenerating the PlanEngine control class

How we prompted it and what we provided

PlanEngine (**src/lib/domain/engine.ts**) is the class tested in our black-box suite. We deleted it from an isolated copy of the project that had no Git history, so the original could not be consulted, and the submitted code was not affected. One prompt was used. Its core instruction was:

> The control class PlanEngine, implemented in src/lib/domain/engine.ts, has been deleted. Recreate that file from scratch. It must export calculatePlan, which assesses charging stations for a driver's trip and returns a ranked PlanResult with explained exclusions.

The prompt told the agent to stay inside the copy, not to edit tests, to run the tests, and to report the first result. It was allowed three fix rounds. We provided the existing engine tests, our 69 black-box tests with their helper, the Lab 3 class diagram and use case model, and the rest of the source code. We held back our 43 executable basis-path tests to check behaviour the agent had not seen.

Iteration

None was needed. The agent worked from the single prompt, ran the tests once, and used no fix rounds. We did not try the fallback of providing the file with only its function bodies removed.

Outcome

Check	Result
Tests given to the agent, first run	91/91 passed
TypeScript check	No errors
Withheld basis-path tests	29/43 passed, 14 failed
Whole unit suite with the regenerated file	141/155 passed
Similarity to the original	378 lines against 427; 13 of 239 non-trivial lines identical

Seven of the 14 failures were naming or wording only: the code **ROUTE_UNAVAILABLE** instead of NO_ROUTE, two differently worded price warnings, and four differently worded summaries. The other seven were real behaviour gaps:

- An unusable direct route was accepted instead of raising an error.
- Invalid connector power was reported as below minimum power.
- Five warnings were missing: unparseable observation time, observation in the future, unparseable retrieval time, calculation time without an offset, and the Demo Fixture notice.

The most instructive result passed every test but was still wrong. Our design documents did not state the Balanced ranking weights, so the agent chose 40% time, 30% cost, 20% detour and 10% availability after equal weights failed test BB-60. The real weights are 35%, 25%, 25% and 15%. It disclosed this guess and 15 other assumptions in its report.

Did it meet our expectations?

Partly. Passing all provided tests at the first attempt was a strong result, and the code was original rather than copied. It did not reproduce the class, because the agent could only be as precise as its materials. The failures fall almost exactly on rules that existed in the code but not in our tests or diagrams. The regenerated file was discarded and the original kept.

What could raise the chance of success

- Provide the numeric ranking weights and the list of exclusion codes in the design documents.
- Provide the basis-path tests too, or the original file with function bodies removed.
- Review the agent's list of assumptions as carefully as its test results.

Reflection 2 (3.3.3): Generating updated class diagrams

Experiment chosen

We chose the suggested exercise of generating the updated class diagram needed for the Lab 5 submission.

How we prompted it and what we provided

One prompt was used, and the agent was given the current source folder and the Lab 3 class diagram:

> Generate the updated UML class diagram of key classes for the final submission ... show the most important classes, modules and relationships in detail, and omit or simplify minor ones. Keep stereotypes for boundary, control and entity roles. Use only operations and fields that really exist in the current code, and reflect what has changed since the Lab 3 design.

Iteration

There was no follow-up conversation. Without being asked, the agent rendered its Mermaid output to confirm that it parsed and compared member names with the source code.

Outcome

The agent produced one Mermaid diagram with 47 classes and 101 relationships. We confirmed both counts with a separate script and spot-checked key elements against the code, including the new **googleMapsDirectionsUrl** and the **calculatePlan** signature. It correctly found that only one structural change had been committed since Lab 3, the Google Maps link. Its other edits corrected the earlier diagram: 10 missing Recommendation fields, private functions shown as public, and absent classes such as ProviderError. It also reported that the Lab 3 diagram did not parse in Mermaid 11.4.1.

Did it meet our expectations?

For accuracy, yes; for usefulness as submitted, not yet. We asked for key classes, but it produced a near-complete inventory that renders at about 10,900 by 6,600 units and is too large to read on a page. Its boundary, control and entity labels are also judgement calls. For example, it labels PlugPlanApp as boundary although that component contains much control logic. Its member check confirmed only that each name exists somewhere in the source, not that it belongs to the class shown. The diagram is therefore a sound draft for Lab 5, not a finished work product.

What could raise the chance of success

- Set an explicit limit, such as at most 15 classes per diagram, and request one diagram per use case.
- Supply our intended boundary, control and entity assignments rather than leaving them to the agent.